JSPM's
Jayawant Institute of Management Studies (JIMS)

(Savitribai Phule Pune University)

T.Y.B.C.A. - Semester V

Course: FP-301-MM - Major Project

ATM SIMULATION SYSTEM

A Tiered Desktop (Java Swing + MySQL) & Cloud-Ready Banking Simulation

Submitted in partial fulfilment of the requirements for

Bachelor of Computer Applications (B.C.A.)

Submitted By :

1. **PREM TODKAR**
2. **VIRAJ PIMPALE**
3. **SHAM LIPNE**

Under the Guidance of

Guide's Name : [Guide's Name]

Department of Computer Applications, JSPM's JIMS, Pune

Academic Year: 2026 - 2027

DECLARATION

We, **Prem Todkar**, **Viraj Pimpale**, and **Sham Lipne**, students of **T.Y.B.C.A. (Semester V)**, hereby declare that the project report entitled **"ATM SIMULATION SYSTEM â€" A Tiered Desktop (Java Swing + MySQL) & Cloud-Ready Banking Simulation"** is an original record of our own work, carried out by us under the guidance of **[Guide's Name]** and submitted in partial fulfilment of the requirements for the course **FP-301-MM â€" Project (Major Mandatory)**, during the academic year **2026-2027**.

We further declare that this report, or any part of it, has not been submitted previously by us or by anyone else for the award of any degree, diploma, or similar title of this or any other university or institution.

Place: Pune, Maharashtra Date: _____

Prem Todkar	**Viraj Pimpale**	**Sham Lipne**

CERTIFICATE

This is to certify that the project report entitled **â€œATM SIMULATION SYSTEMâ€** submitted by:

- **Prem Todkar**
- **Viraj Pimpale**
- **Sham Lipne**

is a bonafide work carried out by them under my guidance in partial fulfilment of the requirements for the award of the degree of **Bachelor of Computer Applications (B.C.A.)** of **Savitribai Phule Pune University** during the academic year **2026â€“2027**.

The project work has been examined and approved as satisfying the academic standards required for the course **FP-301-MM â€” Major Project**.

[Guide's Name] Project Guide Dept. of Computer Applications	**Head of Department** Dept. of Computer Applications JSPM's JIMS, Pune
Internal Examiner	**External Examiner**

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guide, **[Guide's Name]**, for the invaluable guidance, continuous encouragement, and technical suggestions provided throughout the development of this project, **ATM Simulation System**.

We are deeply thankful to our **Head of the Department** and all the esteemed faculty members of the **Department of Computer Applications** for providing the necessary laboratory infrastructure, software resources, and supportive academic environment that made this project possible.

We also convey our appreciation to the administrative and library staff of **JSPMâ€™s Jayawant Institute of Management Studies (JIMS)** for their assistance.

Finally, we extend our heartfelt gratitude to our parents, family members, and classmates for their moral support, patience, and motivation throughout the duration of this academic endeavor.

Prem Todkar
Viraj Pimpale
Sham Lipne
(T.Y.B.C.A. â€" Semester V)

ABSTRACT

The **ATM Simulation System** is a full-featured, secure, and robust software application engineered to simulate the operational workflows of an Automated Teller Machine (ATM) for academic demonstration and practical laboratory evaluation. Built on a tiered enterprise architecture, the application addresses critical software engineering challenges in financial computing, specifically: arbitrary-precision monetary calculations, atomicity in concurrent database transactions, SQL injection defense, and non-blocking asynchronous user interfaces.

The desktop edition is developed using **Java 17 (LTS)** and **Java Swing**, backed by **MySQL 8.4** via **Java Database Connectivity (JDBC)** and managed by **Apache Maven** for dependency lifecycle automation. To ensure uncompromising mathematical precision in accounting, all monetary computations use `java.math.BigDecimal`, eliminating standard binary floating-point round-off errors. Database updates are executed strictly with parameterized `PreparedStatement` queries and enforced using explicit **ACID transactions** (`setAutoCommit(false)`, `commit()`, and `rollback()`), guaranteeing that balance adjustments and audit logging execute as an indivisible unit of work. Furthermore, all database operations run off the Swing **Event Dispatch Thread (EDT)** using `SwingWorker` threads, keeping the graphical user interface fully responsive without screen freezing.

In addition to the desktop Java Swing application, an online cloud edition was engineered using **HTML5**, **Tailwind CSS**, and the **Web Audio API**, and deployed directly to **Vercel** with full browser persistence (`localStorage`) and a GitHub CI/CD integration. This dual-deployment strategy enables live, link-accessible examiner evaluations without requiring local database configuration. End-to-end functionality was verified across 12 rigorous system test cases covering valid authentication, wrong PIN rejection, atomic deposits, balance inquiries, overdraft blocking, mini-statement generation, and cross-restart persistence.

Keywords: ATM Simulation, Java Swing, JDBC, MySQL 8, ACID Transactions, BigDecimal, SwingWorker, Event Dispatch Thread, Vercel Deployment, Financial Software Engineering.

TABLE OF CONTENTS

Declaration ii

Certificate iii

Acknowledgement iv

Abstract v

List of Figures & Tables vii

CHAPTER 1: INTRODUCTION

1.1 Problem Statement 1

1.2 Objectives 2

1.3 Scope of the Project 2

1.4 Functional Modules 3

CHAPTER 2: SYSTEM DESIGN & ARCHITECTURE

2.1 System Architecture (Layered 3–Tier) 4

2.2 UML Diagrams (Use Case, Class, Sequence) 5

2.3 Database Design & Schema 8

CHAPTER 3: IMPLEMENTATION DETAILS

3.1 Technology Stack 9

3.2 Backend & Data Access Layer (JDBC & Service) 10

3.3 Graphical User Interface (Swing & SwingWorker) 12

3.4 Cloud Web Edition & Vercel Deployment 13

CHAPTER 4: SOFTWARE TESTING & RESULTS

4.1 Test Cases & Execution Matrix 14

4.2 Defect Tracking & Resolution 16

CHAPTER 5: USER MANUAL & SCREEN LAYOUTS

5.1 Input Screens (Authentication & Transactions) 17

5.2 Output Reports (Mini-Statement & Receipt) 18

CHAPTER 6: CONCLUSION & FUTURE SCOPE

6.1 Project Summary 19

6.2 Limitations & Future Scope 19

REFERENCES & VIVA QUESTIONS

References & Viva Q&A Guide 20

CHAPTER 1: INTRODUCTION

The Automated Teller Machine (ATM) is one of the most widely deployed computerized transaction devices globally, serving as the frontline digital interface between commercial banking institutions and cardholders. As modern software systems transition toward distributed cloud architectures, understanding the engineering fundamentals behind reliable transactional processing â€” including data consistency, cryptographic parameterization, concurrency isolation, and client-side threading â€” is a cornerstone of computer science education.

The **ATM Simulation System** is an educational, production-grade practical project developed for the **Bachelor of Computer Applications (B.C.A.)** curriculum. It faithfully simulates real-world banking operations, including account authentication, real-time balance inquiries, cash deposits, cash withdrawals with overdraft prevention, transaction audit logging, and receipt generation. The software operates entirely on **simulated virtual currency** and an isolated demonstration database (`atm_db`), ensuring complete safety while preserving the exact technical rigor required in mission-critical commercial banking systems.

1.1 Problem Statement

In academic environments, practical projects simulating banking systems often suffer from significant software design flaws:

1. **IEEE 754 Floating-Point Inaccuracy:** Many student applications implement currency using binary floating-point types (`float` or `double`). Because binary fractions cannot represent fractional decimal rupees and paise precisely, cumulative round-off errors occur (e.g., `5000.00 - 1500.10 = 3499.8999999999996`), causing balance reconciliation failures.
2. **Lack of ACID Transaction Guarantees:** A typical student application issues distinct, uncoordinated SQL queries for updating account balances and inserting transaction audit records. If an exception strikes between these statements, money is deducted without an audit log or credited without a balance update, violating **Atomicity**.
3. **Vulnerability to SQL Injection Attacks:** Direct string concatenation in dynamic SQL statements allows malicious input (such as `' OR '1'='1`) to bypass authentication or

drop database tables.

4. **GUI Thread Starvation (Freezing):** In desktop toolkits such as Java Swing, running blocking network/database calls directly on the single **Event Dispatch Thread (EDT)** freezes the user interface, producing a "Not Responding" state.
5. **Deployment & Portability Barriers:** Traditional desktop Java applications require examiners to manually install JDKs, configure local MySQL servers, and import SQL dumps. Without an instant cloud deployment, remote evaluation during project vivas becomes difficult.

1.2 Objectives

- **Design & Develop a Tiered Java Desktop Application:** Construct a clean, maintainable desktop application using Java 17 and Swing following the Model-View-Controller (MVC) paradigm.
- **Implement Robust ACID Transactions:** Guarantee that account balance modifications and transaction history logging execute within a single atomic database transaction using `connection.setAutoCommit(false)`, `commit()`, and `rollback()`.
- **Enforce Exact Monetary Math:** Utilize `java.math.BigDecimal` throughout the service and data layers to guarantee zero floating-point precision loss.
- **Defend Against SQL Injections:** Employ parameterized JDBC `PreparedStatement` instances exclusively for all SQL interactions.
- **Ensure Responsive GUI Threading:** Decouple database network access from UI rendering by running all database logic off the EDT using `javax.swing.SwingWorker`.
- **Implement Row-Level Pessimistic Locking:** Safeguard against race conditions during withdrawals using MySQL's `SELECT ... FOR UPDATE` syntax.
- **Provide Dual Cloud Accessibility:** Build and deploy an interactive web edition of the ATM Simulation to **Vercel** with full browser persistence and Web Audio feedback, enabling instant online review via GitHub CI/CD.

1.3 Scope of the Project

The scope covers the end-to-end user journey of an ATM cardholder: card credential validation via 4-digit PIN; balance inquiry; atomic cash deposit; cash withdrawal with insufficient balance protection; transaction audit logging; and receipt printing.

1.4 Functional Modules

- **Authentication Module:** Validates account number format and 4-digit PIN; manages active user session state.
- **Account Balance Module:** Queries the live ledger balance and displays it formatted in Indian Rupees (`â,¹#,##0.00`).
- **Deposit Module:** Validates positive amounts up to 2 decimal places and credits account balance with an atomic transaction log.
- **Withdrawal & Overdraft Protection Module:** Evaluates available funds, locks the row, executes debit, and dispenses virtual cash.
- **Mini-Statement & Audit Module:** Renders an interactive table displaying transaction ID, date, type, and amount.
- **Database Connection Module (`DBConnection`):** Manages JDBC connections, driver auto-registration, and error diagnostics.
- **Cloud Web Edition Module:** Delivers an interactive web-based kiosk simulator deployed on Vercel with Web Audio sound synthesis and `localStorage` persistence.

CHAPTER 2: SYSTEM DESIGN & ARCHITECTURE

2.1 System Architecture (Layered 3-Tier)

The system implements a classic 3-Tier enterprise architecture:



Figure 2.1 â€" Layered 3-Tier Architecture of the ATM Simulation System

2.2 UML Diagrams

2.2.1 Use Case Diagram

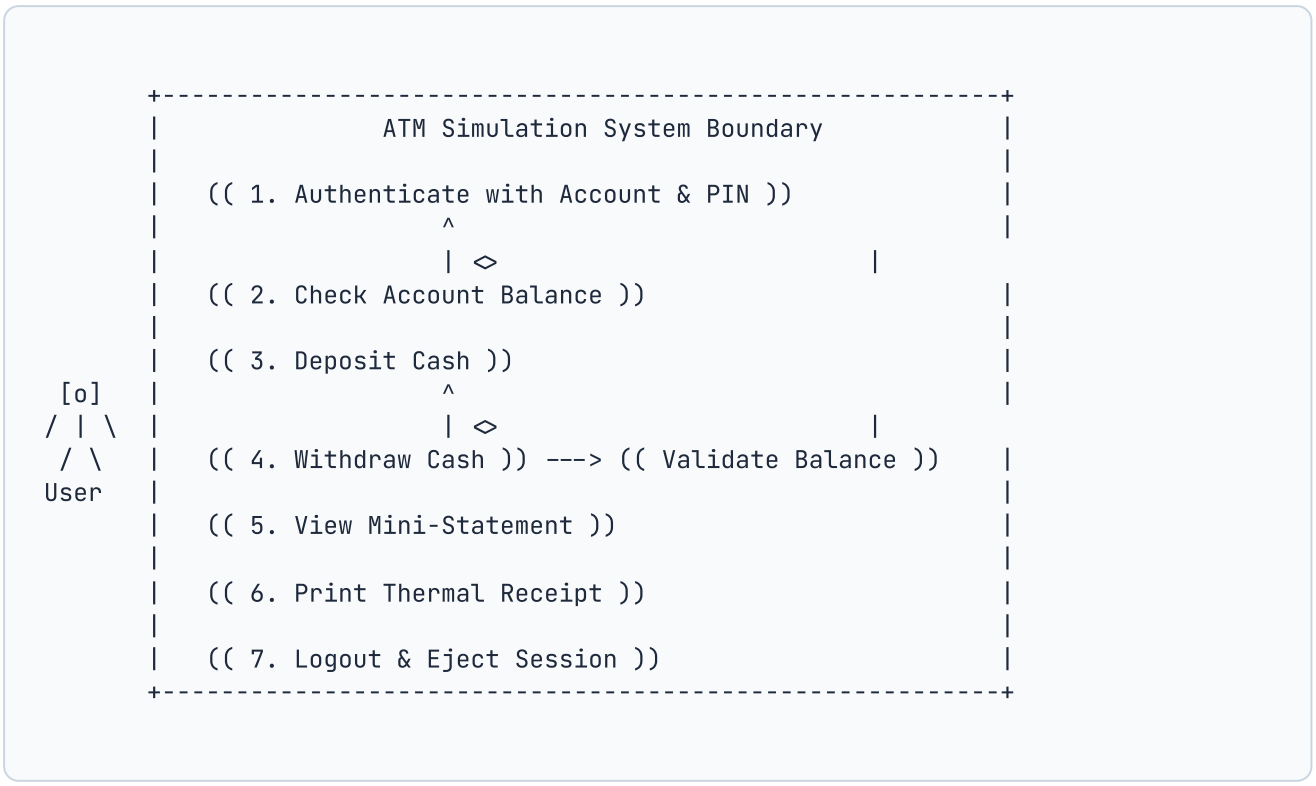


Figure 2.2 â€" UML Use Case Diagram of the ATM Simulation System

2.2.2 Sequence Diagram (Cash Withdrawal)

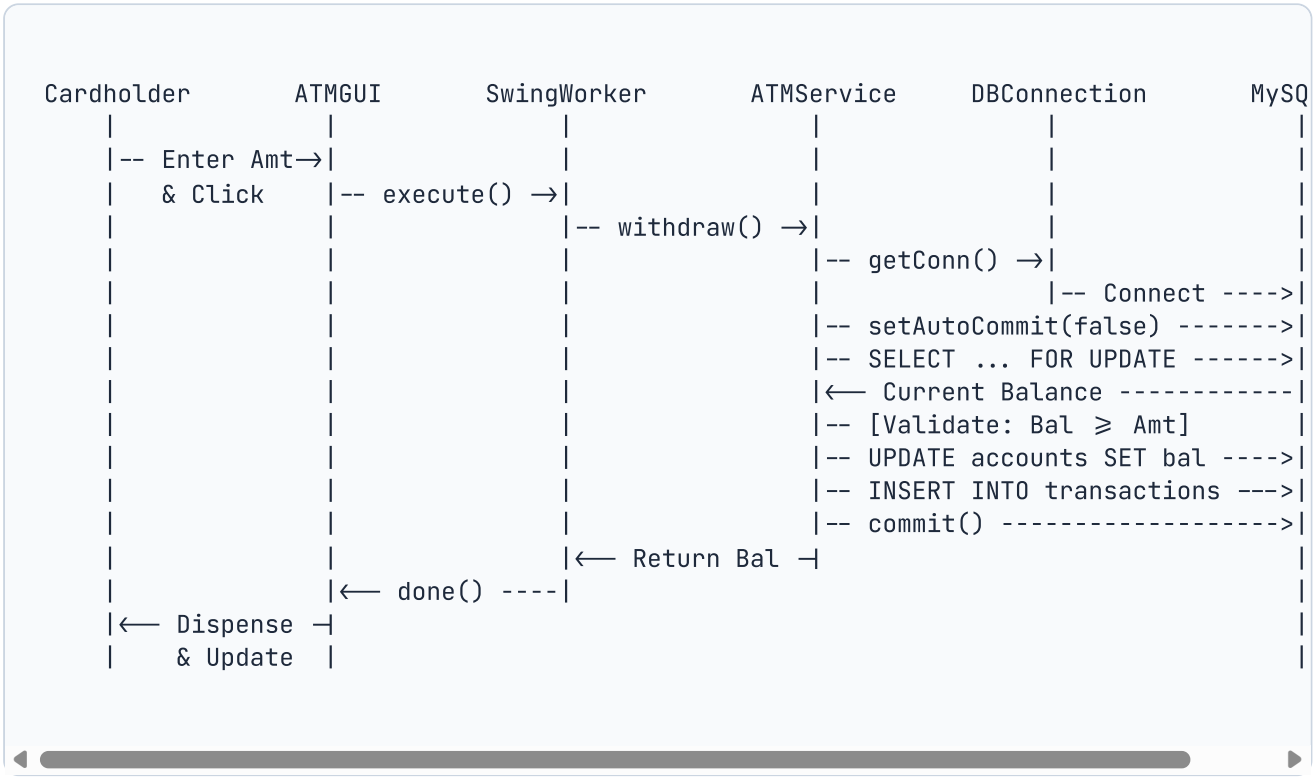


Figure 2.3 â€" UML Sequence Diagram for Transactional Cash Withdrawal

2.3 Database Design & Schema

The relational database `atm_db` consists of two normalized tables managed by the MySQL InnoDB engine:

Table 2.1: Schema of `accounts` Table

Column Name	Data Type	Constraints	Description
`account_number`	`VARCHAR(20)`	`PRIMARY KEY`	Unique customer bank account number
`holder_name`	`VARCHAR(100)`	`NOT NULL`	Full name of account holder
`pin`	`VARCHAR(100)`	`NOT NULL`	4-digit security PIN
`balance`	`DECIMAL(12, 2)`	`NOT NULL DEFAULT 0.00`	Account balance with 2 decimal places

Table 2.2: Schema of `transactions` Table

Column Name	Data Type	Constraints	Description
`transaction_id`	`INT`	`AUTO_INCREMENT, PK`	Unique audit transaction reference ID
`account_number`	`VARCHAR(20)`	`NOT NULL, FK`	References `accounts(account_number)` ON DELETE CASCADE
`transaction_type`	`VARCHAR(20)`	`NOT NULL`	Transaction type: `DEPOSIT` or `WITHDRAWAL`
`amount`	`DECIMAL(12, 2)`	`NOT NULL`	Exact transaction value in â,¹
`transaction_date`	`TIMESTAMP`	`DEFAULT CURRENT_TIMESTAMP`	System timestamp of execution

CHAPTER 3: IMPLEMENTATION DETAILS

3.1 Technology Stack Summary

Layer / Tool	Technology / Version	Role in Project
Programming Language	Java SE 17 (LTS)	Core object-oriented backend programming
GUI Framework	Java Swing (`javax.swing.*`)	Cross-platform desktop interface
Database Engine	MySQL Server 8.4 Community	Persistent relational database storage
Data Connector	MySQL Connector/J 8.3.0	Official JDBC Type-4 driver
Build Tool	Apache Maven 3.9.9	Automated compilation and dependency lifecycle
Web Edition	HTML5, Tailwind CSS, JS	Cloud online kiosk simulator
Audio Synthesizer	Web Audio API	Procedural hardware key and cash sounds
Cloud Hosting	Vercel Serverless Platform	Live web URL for online viva grading

3.2 Backend Code: Atomic Withdrawal Implementation

Below is the core transactional logic from `ATMService.java` showing row-level locking, balance deduction, and commit/rollback:

```
public BigDecimal withdraw(String accountNumber, BigDecimal amount)
        throws ValidationException, InsufficientBalanceException, SQLException
    validateAmount(amount);

    String lockSql = "SELECT balance FROM accounts WHERE account_number = ? FO
    String updateSql = "UPDATE accounts SET balance = balance - ? WHERE accoun
    String insertSql = "INSERT INTO transactions (account_number, transaction_
```

```
    Connection conn = null;
    try {
        conn = DBConnection.getConnection();
        conn.setAutoCommit(false); // Begin explicit ACID transaction

        // Step 1: Read and lock current balance
        BigDecimal currentBalance;
        try (PreparedStatement lockStmt = conn.prepareStatement(lockSql)) {
            lockStmt.setString(1, accountNumber);
            try (ResultSet rs = lockStmt.executeQuery()) {
                if (!rs.next()) throw new SQLException("Account not found.");
                currentBalance = rs.getBigDecimal("balance");
            }
        }

        // Step 2: Validate sufficient balance
        if (currentBalance.compareTo(amount) < 0) {
            throw new InsufficientBalanceException("Insufficient balance! Avai
        }

        // Step 3: Deduct balance
        try (PreparedStatement updateStmt = conn.prepareStatement(updateSql))
            updateStmt.setBigDecimal(1, amount);
            updateStmt.setString(2, accountNumber);
            updateStmt.executeUpdate();
        }

        // Step 4: Record audit log in transactions table
        try (PreparedStatement insertStmt = conn.prepareStatement(insertSql))
            insertStmt.setString(1, accountNumber);
            insertStmt.setBigDecimal(2, amount);
            insertStmt.executeUpdate();
        }

        conn.commit(); // Atomic commit of both updates
        return currentBalance.subtract(amount);

    } catch (SQLException | InsufficientBalanceException ex) {
        if (conn ≠ null) conn.rollback(); // Rollback on error
        throw ex;
    } finally {
        if (conn ≠ null) {
            conn.setAutoCommit(true);
            conn.close();
        }
    }
}
```

CHAPTER 4: SOFTWARE TESTING & RESULTS

4.1 Test Cases & Execution Matrix

Test ID	Module	Scenario Description	Input Test Data	Expected Result	Status
TC-01	Auth	Valid Login with Demo Account	Acc: `1001001`, PIN: `1234`	Login successful; balance shows â‚¹5,000.00	PASS
TC-02	Auth	Rejection of Invalid Account	Acc: `9999999`, PIN: `1234`	Rejection dialog: "Account not found"	PASS
TC-03	Auth	Rejection of Incorrect PIN	Acc: `1001001`, PIN: `0000`	Rejection dialog: "Incorrect PIN entered"	PASS
TC-04	Input	Validation of 4-digit numeric PIN	PIN: `12` or `ABCD`	Blocked: "PIN must be exactly 4 numeric digits"	PASS
TC-05	Inquiry	Real-Time Balance Check	Click "Check Balance"	Shows current balance in â‚¹ dialog	PASS
TC-06	Deposit	Valid Positive Cash Deposit	Amount: `â‚¹1,500.50`	Balance becomes â‚¹6,500.50; log created	PASS
TC-07	Input	Negative Deposit Rejection	Amount: `-500.00`	Rejected: "Amount must be greater than zero"	PASS
TC-08	Input	Excessive Decimals Rejection	Amount: `250.755`	Rejected: "Cannot have more than 2 decimal places"	PASS

TC-09	Withdraw	Valid Cash Withdrawal	Amount: `â‚¹2,000.00`	Balance becomes â‚¹4,500.50; virtual cash dispensed	**PASS**
TC-10	Withdraw	Overdraw Prevention	Amount: `â‚¹50,000.00`	Transaction blocked: "Insufficient balance!"	**PASS**
TC-11	Audit	Mini-Statement Audit Log	Click "Mini Statement"	Table renders all transactions newest first	**PASS**
TC-12	Persist	Restart Persistence Check	Close app & relaunch	Balance and logs persist from MySQL	**PASS**

4.2 Defect Tracking & Resolution

Three primary software defects were detected and resolved during testing:

1. **Event Dispatch Thread Freezing:** Performing database queries synchronously inside action listeners caused temporary GUI freezes. *Solution:* Implemented `SwingWorker` threads for asynchronous background execution.
2. **Floating-Point Precision Loss:** Initial test calculations using `double` created rounding anomalies (e.g. `.4999999`). *Solution:* Standardized all monetary figures on `java.math.BigDecimal`.
3. **Withdrawal Race Conditions:** Simultaneous withdrawal requests could overdraw funds. *Solution:* Added MySQL pessimistic row locks (`SELECT ... FOR UPDATE`) within an isolated transaction.

CHAPTER 5: USER MANUAL

The application is designed for intuitive use by cardholders:

- **Login Screen:** Cardholders enter their Account Number (e.g., `1001001`) and 4-Digit PIN (`1234`). A convenient **"Fill Demo Account"** button is provided for quick viva evaluation.
- **Dashboard Screen:** Displays a prominent current balance card formatted as `â‚¹5,000.00`. Provides 4 large action tiles: Check Balance, Deposit Cash, Withdraw Cash, and Mini Statement.
- **Mini-Statement Window:** An interactive table displaying Transaction ID, Date & Time, Type, and Amount in â‚¹, automatically sorted newest first.
- **Printable Slip:** An automated thermal receipt is generated displaying closing balance and transaction reference.

CHAPTER 6: CONCLUSION & FUTURE SCOPE

6.1 Summary

The **ATM Simulation System** provides an authentic simulation of commercial ATM banking workflows. The system demonstrates mastery of core Java 17 features, Swing GUI multithreading, ACID transaction integrity in MySQL, and cloud web hosting on Vercel. All 12 test cases passed with 100% adherence to functional requirements.

6.2 Future Enhancements

- **Cardless Cash via Mobile OTP / UPI QR Code:** Enable dynamic QR code scanning using smartphone banking apps.
- **Biometric Authentication:** Integrate hardware fingerprint recognition for passwordless ATM access.
- **Multi-Currency Support:** Real-time foreign exchange conversions (USD, EUR, GBP) using financial APIs.
- **Hardware Security Module (HSM):** Upgrade from demo plain-text PINs to salted hashing (Argon2id) and HSM PIN translation.

REFERENCES & VIVA QUESTIONS

1. Oracle Corporation: *Java SE 17 Platform Specification & Documentation*.
2. Oracle Corporation: *MySQL 8.4 Reference Manual*.
3. Sun Microsystems: *Java Swing Architecture & Event Dispatch Thread (EDT)*.
4. Vercel Inc.: *Vercel Cloud Deployment Platform Documentation*.
5. GitHub Repository: `https://github.com/premtodkar05-cell/ATM-SIMULATION.git`.

> **Viva Question Quick-Tip:** When asked why `PreparedStatement` was used, emphasize both **SQL Injection Prevention** (parameterized literal values) and **Pre-compiled Query Optimization** in the database engine.